

LEGO Database Schema Description

October 2, 2026

Terminologie pro česká zadání

V českých zadáních používáme pro tabulky tyto české názvy. Názvy tabulek a sloupců v SQL zůstávají anglické.

- **sets:** stavebnice
- **themes:** témata; kořenové téma je téma s `parent_id IS NULL`
- **inventories:** inventáře; jeden inventář je jedna verze inventáře stavebnice
- **inventory_sets:** další stavebnice obsažené v inventáři
- **minifigs:** minifigurky
- **inventory_minifigs:** minifigurky v inventáři
- **part_categories:** kategorie dílků
- **parts:** dílky (jeden dílek)
- **inventory_parts:** dílky v inventáři
- **colors:** barvy
- **elements:** elementy (jeden element)
- **part_relationships:** vztahy mezi dílky, jeden dílek je vždy rodičovský a druhý je potomek.

V `inventory_parts` znamená `is_spare = TRUE` náhradní dílek a `is_spare = FALSE` běžný dílek.

Entity-Relationship Diagram (ERD)

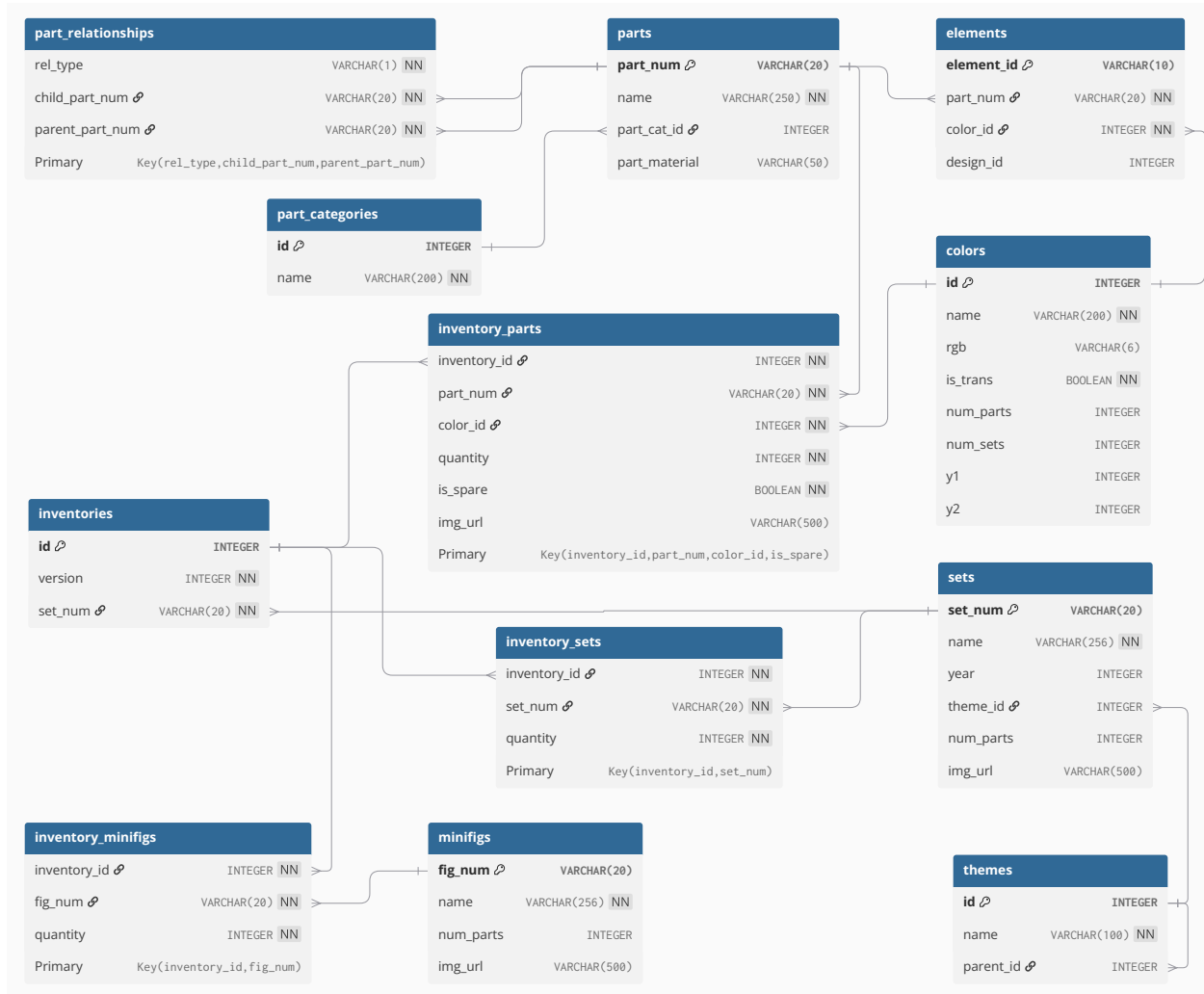


Table: sets (stavebnice)

Schema

Attribute	Type
set_num	string
name	string
year	integer
theme_id	integer
num_parts	integer
img_url	string

Meaning of a Record

Each record in the **sets** table represents a specific LEGO set available in the database. It contains all the essential information about a particular set including its identification number, name, release year, associated theme, and other metadata.

Attribute Explanations

- **set_num**: Unique identifier for each LEGO set. This is a string that typically follows a specific format (e.g., "0003977811-1") which helps identify the exact set.
- **name**: The name of the LEGO set, often reflecting its theme or concept (e.g., "Ninjago: Book of Adventures").
- **year**: The year when the LEGO set was first released or made available to the public.
- **theme_id**: Foreign key that links this set to a specific theme in the **themes** table. This establishes the thematic category of the set.
- **num_parts**: The number of individual LEGO parts (dílků) included in this set. A value of 0 may indicate a promotional or bonus set with no physical dílky.
- **img_url**: A URL pointing to an image of the LEGO set, allowing for visual representation in applications.

Table: themes (témata)

Schema

Attribute	Type
id	integer
name	string
parent_id	integer

Meaning of a Record

Each record in the **themes** table represents a category or theme that LEGO sets belong to. These themes help organize and classify different types of LEGO sets, such as specific series or concepts. The themes form a hierarchical structure where some themes are sub-themes of others.

Attribute Explanations

- **id**: Primary key for the theme table. This unique identifier is used to reference this specific theme throughout the database.
- **name**: The name of the LEGO theme (e.g., "Technic", "Competition", "Expert Builder"). This represents the category or concept that the theme belongs to.
- **parent_id**: Foreign key that references another theme in the same table, creating a hierarchical structure. If this value is NULL, this is a root theme (kořenové téma), meaning a theme without a parent. If it has a value, it indicates that this theme is a sub-theme of another theme (e.g., "Competition" under "Technic").

Tree Model and Hierarchical Structure

The themes table implements a tree data structure where each theme can have a parent theme. This hierarchical model allows for organizing LEGO sets in multiple levels:

- **Root Themes (kořenová témata)**: These are themes with **parent_id** IS NULL, representing the main categories (e.g., "Technic", "City", "Creator").

```
SELECT name FROM themes WHERE parent_id IS NULL;
```

- **Child Themes**: These are themes that belong to a parent theme, creating a multi-level hierarchy.

```
SELECT t1.name FROM themes t1
JOIN themes t2 ON t1.parent_id = t2.id
WHERE t2.name = 'Technic';
```

Table: inventories (inventáře)

Schema

Attribute	Type
id	integer
version	integer
set_num	string

Meaning of a Record

Each record in the `inventories` table represents one inventory version intended for a LEGO set. One set can have multiple inventory records for different versions, so the relationship from `sets` to `inventories` is one-to-many, not one-to-one. In the loaded data, 1,280 `set_num` values occur in more than one inventory row; one value occurs in as many as 12 rows.

Attribute Explanations

- **id:** Primary key of the inventory. This identifier is used by related tables such as `inventory_parts`, `inventory_minifigs`, and `inventory_sets`.
- **version:** Version number of the inventory for the given set. Different versions can represent changes or corrections in the recorded contents of the set.
- **set_num:** Non-null identifier of the LEGO set whose inventory is intended to be described. It should correspond to `set_num` in the `sets` table, but the DDL script keeps this foreign key commented out because some imported inventory records refer to set numbers absent from `sets`. The loaded data has 17,223 such inventory rows. Therefore, this intended relationship is not fully enforced by the database.

Table: `inventory_sets` (další stavebnice v inventáři)

Schema

Attribute	Type
<code>inventory_id</code>	integer
<code>set_num</code>	string
<code>quantity</code>	integer

Meaning of a Record

Each record in the `inventory_sets` table points to an additional inventory that is related to a LEGO set.

Attribute Explanations

- **`inventory_id`**: Foreign key referencing `id` in the `inventories` table. It identifies an inventory associated with the LEGO set.
- **`set_num`**: Foreign key referencing `set_num` in the `sets` table. It identifies the LEGO set to which the additional inventories are associated.
- **`quantity`**: Number of copies of the additional inventory associated with the set.

Relationship to Inventories and Sets

The following query returns sets that are contained in set '66214-1'

```
SELECT i.id, i.set_num, s.*
FROM inventories i
      JOIN inventory_sets ins ON i.id = ins.inventory_id
      JOIN sets s ON ins.set_num = s.set_num
WHERE i.set_num = '66214-1';
```

Table: minifigs (minifigurky)

Schema

Attribute	Type
fig_num	string
name	string
num_parts	integer
img_url	string

Meaning of a Record

Each record in the `minifigs` table represents a particular LEGO minifigure. The table is a catalog of minifigures that can occur in set inventories. A minifigure record describes the figure itself, not its occurrence in a particular set.

Attribute Explanations

- **fig_num**: Primary key and unique identifier of the minifigure design, for example `fig-000001`.
- **name**: Name or descriptive label of the minifigure, such as “Toy Store Employee” or “Wonder Woman”.
- **num_parts**: Number of LEGO parts (dílků) used to construct the minifigure. This describes the minifigure design and is different from the number of copies of the figure in an inventory.
- **img_url**: URL of an image showing the minifigure.

Table: `inventory_minifigs` (minifigurky v inventáři)

Schema

Attribute	Type
<code>inventory_id</code>	integer
<code>fig_num</code>	string
<code>quantity</code>	integer

Meaning of a Record

Each record in the `inventory_minifigs` table records how many copies of a particular minifigure are present in one inventory. For example, the sample row (26, `fig-008641`, 1) means that inventory 26 contains one copy of minifigure `fig-008641`.

Attribute Explanations

- **`inventory_id`**: Foreign key referencing `id` in the `inventories` table. It identifies the inventory of a LEGO set.
- **`fig_num`**: Foreign key referencing `fig_num` in the `minifigs` table. It identifies the minifigure included in the inventory.
- **`quantity`**: Number of copies of the minifigure in the inventory.

Relationship to Inventories and Minifigs

The `inventory_minifigs` table is a binding table between `inventories` and `minifigs`. One inventory can contain multiple minifigure designs, and the same minifigure design can occur in multiple inventories. The composite primary key (`inventory_id`, `fig_num`) allows each minifigure to occur at most once per inventory; the `quantity` attribute stores how many copies are present.

Table: `part_categories` (kategorie dílků)

Schema

Attribute	Type
id	integer
name	string

Meaning of a Record

Each record in the `part_categories` table represents a category used to classify LEGO parts (dílků). The sample data includes categories such as “Bricks”, “Minifigs”, “Minifig Accessories”, “Animals / Creatures”, and “Stickers”. A category can be assigned to many dílky.

Attribute Explanations

- **id**: Primary key and unique identifier of the part category (kategorie dílků).
- **name**: Name of the category used to group similar dílky.

Table: parts (dílky)

Schema

Attribute	Type
part_num	string
name	string
part_cat_id	integer
part_material	string

Meaning of a Record

Each record in the **parts** table represents a LEGO part (dílek). Its name describes its form, function, or printed design. For example, the sample data contains printed minifigure heads, animal parts, medical equipment, helmets, hipwear, and sticker sheets. A dílek can occur in many inventories.

Attribute Explanations

- **part_num**: Primary key and unique identifier of the dílek, for example 01920pr0001 or 02135.
- **name**: Descriptive name of the dílek, including information about its type, appearance, or printed pattern.
- **part_cat_id**: Foreign key referencing **id** in the **part_categories** table. It classifies the dílek, for example as a minifig head, minifig headwear, animal or creature, or sticker.
- **part_material**: Material of the dílek, such as **Plastic** or **Rubber**.

Table: `inventory_parts` (dílký v inventáři)

Schema

Attribute	Type
<code>inventory_id</code>	integer
<code>part_num</code>	string
<code>color_id</code>	integer
<code>quantity</code>	integer
<code>is_spare</code>	boolean
<code>img_url</code>	string

Meaning of a Record

Each record in the `inventory_parts` table records a particular dílek, in a particular color and inventory. It also records how many copies are present and whether they are spare dílky. For example, the sample row `(129068, 01920pr0001, 484, 1, false, ...)` means that inventory 129068 contains one non-spare copy of dílek 01920pr0001 in color 484.

Attribute Explanations

- **`inventory_id`**: Foreign key referencing `id` in the `inventories` table. It identifies the inventory in which the dílek occurs.
- **`part_num`**: Foreign key referencing `part_num` in the `parts` table. It identifies the dílek included in the inventory.
- **`color_id`**: Foreign key referencing `id` in the `colors` table. It identifies the color of the dílek in this inventory record.
- **`quantity`**: Number of copies of the dílek in the inventory.
- **`is_spare`**: Indicates whether the recorded dílek is a spare (náhradní dílek). `TRUE` means a náhradní dílek; `FALSE` means a běžný dílek. The default value is `FALSE`.
- **`img_url`**: URL of an image representing the dílek in the recorded color or inventory context.

Relationship to Inventories, Parts, and Colors

The `inventory_parts` table is a binding table connecting `inventories`, `parts`, and `colors`. One inventory can contain many dílky, and one dílek can occur in many inventories and colors. The composite primary key (`inventory_id`, `part_num`, `color_id`, `is_spare`) permits separate records for different colors or for spare and regular copies of the same dílek. The `quantity` attribute stores the number of copies represented by each record.

Table: colors (barvy)

Schema

Attribute	Type
id	integer
name	string
rgb	string
is_trans	boolean
num_parts	integer
num_sets	integer
y1	integer
y2	integer

Meaning of a Record

Each record in the `colors` table represents a LEGO color that can be assigned to dílky. The catalog includes regular colors such as Black, Blue, Green, Red, and Light Gray, as well as an “Unknown” color with identifier -1. The table also stores usage statistics and the period in which the color was used.

Attribute Explanations

- **id**: Primary key and unique identifier of the color. The value -1 is used for the “Unknown” color in the sample data.
- **name**: Human-readable name of the color, such as Black, Blue, or Dark Turquoise.
- **rgb**: Six-character RGB hexadecimal code representing the color, for example 05131D for Black.
- **is_trans**: Indicates whether the color is transparent. The sample rows shown are opaque and therefore have the value `FALSE`.
- **num_parts**: Number of dílků in the database associated with this color.
- **num_sets**: Number of sets in the database that use this color.
- **y1**: First year in which the color is recorded as being used.
- **y2**: Last year in which the color is recorded as being used.

Table: elements (elementy)

Schema

Attribute	Type
element_id	string
part_num	string
color_id	integer
design_id	integer

Meaning of a Record

Each record in the `elements` table represents one element: a specific colored version of a dílek. It connects a dílek design with a color and can optionally provide a design ID for a decorated or otherwise specialized element. For example, element 4190230 refers to dílek 973c26h01pr0391 in color 19.

Attribute Explanations

- **element_id**: Primary key and unique identifier of the physical element or element-color variant.
- **part_num**: Foreign key referencing `part_num` in the `parts` table. It identifies the underlying dílek design.
- **color_id**: Foreign key referencing `id` in the `colors` table. It identifies the color of the element.
- **design_id**: Optional design identifier for a decorated or specialized element. It may be NULL when no separate design identifier is recorded. The table referenced by this attribute is missing in the current database.

Relationship to Parts and Colors

The `elements` table provides a more specific identification than the `parts` table alone. A dílek can have multiple elements because the same dílek design can be available in different colors or with different designs. Each element references exactly one dílek and one color, while the optional `design_id` can distinguish a decorated variant.

Rows in `inventory_parts` do not reference `element_id`; they identify a dílek and color using `part_num` and `color_id`. To associate an inventory row with an element, match both of these columns. If multiple element rows share the same dílek and color, the inventory data does not distinguish between those elements; they therefore have the same inventory counts for that dílek and color.

Table: `part_relationships` (vztahy mezi dílky)

Schema

Attribute	Type
<code>rel_type</code>	string
<code>child_part_num</code>	string
<code>parent_part_num</code>	string

Meaning of a Record

Each record in the `part_relationships` table describes a relationship between two LEGO parts (dílky). The `child_part_num` and `parent_part_num` columns both reference the `parts` table; the relationship is therefore between dílky in the same catalog. The meaning of the relationship are specified by `rel_type`.

Attribute Explanations

- **`rel_type`:** Code identifying the type of relationship. The data contains the following types: A (alternate), B (base), M (mold), P (print), R (replacement), and T (tool).
- **`child_part_num`:** Foreign key referencing `part_num` in the `parts` table. It identifies the related dílek on the child side of the relationship (dílek na straně child).
- **`parent_part_num`:** Foreign key referencing `part_num` in the `parts` table. It identifies the rodičovský dílek, on the parent side of the relationship.

Relationship Types in the Data

The sample aggregation shows the distribution of relationship types:

Type	Number of Records
A (alternate)	1,180
B (base)	1,559
P (print)	30,040
T (tool)	461
M (mold)	1,161
R (replacement)	2,987

Keys and Referential Integrity

The composite primary key (`rel_type`, `child_part_num`, `parent_part_num`) prevents the same relationship from being stored more than once for a given type. Both part-number columns have foreign keys to `parts`, so every related child and parent dílek must exist in the dílek catalog.